{EPITECH}

AREA_

< ADDITIONAL FEATURES />

AREA

This document holds the detail of the feature catalogue indexed in the subject.



Security work must be demonstrated, not only described. For each selected **SEC** feature, keep a short evidence note covering the attack scenario, how to reproduce it, what you observed, the protection and where it lives, the protected result, and its limitations. A suggested structure is provided.



Some acceptance criteria are satisfied by a document rather than by a running program. In such cases, the document must contain claims that can be demonstrated live.
A claim that cannot be demonstrated does not satisfy the criterion, and a document that nobody in the group can defend is not a deliverable.

AI-EVAL - Model evaluation

In our customer's words:
”Measure the behavior you claim instead of calling it intelligent.”

What your boss defined as done:

- ✓ A committed labelled set contains at least fifteen examples and documents how, and by whom, labels were produced.
- ✓ A reproducible command computes a metric appropriate to the evaluated use case, and the evaluator runs it.
- ✓ The measurement runs against the deterministic implementation as a stable control.
- ✓ At least two dated measurements make a change visible and identify the evaluated model.
- ✓ The documentation connects the client need, the implementation, the metric and its limitations, including weaknesses of the labelled set.



`AI-EVAL` requires `AI-NODE`; both count toward the selection.

{EPITECH}

AI-GUARD - Prompt injection defense

In our customer's words:
”Text written outside my company must not steer your robot, and my employees' names must not reach the model provider.”

What your boss defined as done:

- ✓ A committed corpus of at least twenty attack payloads states the effect each one attempts. One command runs it through the node and reports, per payload, that the declared keys are unchanged and that the effect did not happen.
- ✓ Third party content is never treated as an instruction. The evaluator follows one payload to the side of the boundary it lands on.
- ✓ The personal data categories the node removes are a fixed, documented list held in the node configuration.
- ✓ Three planted identifiers of those categories are absent from the outbound request, which the evaluator reads before it leaves.
- ✓ The node output restores the three identifiers byte for byte.
- ✓ Blocked attempts are recorded with their date and originating **AREA**.
- ✓ Everything is demonstrated on an implementation returning its input unchanged, with no model called. `AI-NODE` is not required, and a deterministic implementation shared with it counts once.

AI-MULTIMODAL - Multimodal inputs

In our customer's words:
”Turn my field team's voice notes and photos into something the automation can read.”

What your boss defined as done:

- ✓ A node turns a sound recording or an image produced by an earlier node into text published through the ordinary key path mechanism, and a later node consumes it.
- ✓ The documented representation of payloads that are not text matches the data model.
- ✓ A deterministic implementation returns the same text twice for the same input, with no model called. `AI-NODE` is not required, and a deterministic implementation shared with it counts once.
- ✓ Stated size and duration limits refuse an oversized payload with a reason naming the limit.
- ✓ An unsupported format is refused at the node and ends the run in the documented state, without bringing the chain down.
- ✓ Payload retention and location are stated, and verifiable when the payload is kept.

AI-NODE - Autonomous AI node

In our customer's words:
”Use a language model as a workflow capability without making the whole engine depend on one provider.”

What your boss defined as done:

- ✓ An AI node participates in the ordinary workflow and key path mechanism.
- ✓ Model access sits behind an abstraction with a real implementation and a deterministic one, and switching between them is configuration.
- ✓ Deterministic runs are reproducible with nothing leaving the machine. The evaluator runs the same workflow twice and compares.
- ✓ Cost is bounded by documented controls such as quota, timeout and size limits, each demonstrated with what the user sees when it is reached.
- ✓ Repeated identical work is served without a second outbound call.
- ✓ A dry run exposes what would be requested without sending it.
- ✓ At least two use cases have declared input and output shapes and credible user stories.
- ✓ Published keys are declared per instance, and a key path to an undeclared key is refused when the workflow is saved.

AI-PROMPT - Prompt versioning

In our customer's words:
”When a prompt change breaks the sorting, I want the old one back tonight and the two compared.”

What your boss defined as done:

- ✓ Every instruction sent to a model lives in a versioned artifact outside the source code, with a version identifier and a date.
- ✓ The version a node uses is configuration, and a rollback takes effect on the next run without touching the source code.
- ✓ One command runs two versions over a committed set of at least ten inputs and prints the same named measure for each, side by side.
- ✓ The command runs with no model available, on a deterministic implementation this requirement carries. Neither `AI-NODE` nor `AI-EVAL` is required, and a deterministic implementation shared with them counts once.
- ✓ At least two dated comparisons are committed with the version identifiers they compared.



AI-REVIEW - Human validation

In our customer's words:
”Your product may draft the answer, but one of my people approves it, and I want to know what happens when nobody does.”

What your boss defined as done:

- ✓ A review node suspends the run, and nothing placed after it executes.
- ✓ A suspended run survives a restart of the application server.
- ✓ The web client lists pending reviews with the value produced before them, and an approval resumes the chain.
- ✓ A value edited before approval is what the following nodes receive through the ordinary key path mechanism.
- ✓ A rejected review ends the run as rejected, distinguishable from failed.
- ✓ An unanswered review reaches the configured terminal state after the configured delay, which the evaluator sets to one minute.
- ✓ The mobile client lists pending reviews, and acting on them is web only.
- ✓ The node makes no model call and is demonstrated on a chain producing a constant value. `AI-NODE` is not required.

ENG-ADMIN - Administration area

In our customer's words:
”Support must administer the product without opening the database or stealing user credentials.”

What your boss defined as done:

- ✓ An administrator role protects an administration area, and a normal user is refused.
- ✓ Administrators can inspect and disable users and **services**: a disabled user can no longer authenticate, and **AREA** depending on a disabled **service** stop triggering.
- ✓ Administrators can inspect user **AREA** without learning third party tokens.
- ✓ State changing administrative actions are audited with actor and time.
- ✓ The administration area links to execution records rather than duplicating them when `SYS-OBS` is also selected.

ENG-ADR - Architecture decision records

In our customer's words:
”Leave the reasons behind important architecture choices, not a novel.”

What your boss defined as done:

- ✓ At least three significant **architecture decision records** are committed, one of them about the work-

flow engine, each saying why it deserved a record.
- ✓ Each gives context, real alternatives, decision and accepted consequences.
- ✓ Rejected alternatives are rejected for this project, not declared universally bad.
- ✓ Records are dated and carry a status, and superseded decisions point to their replacement.
- ✓ Any group member can say what the product would look like under a rejected option the evaluator picks.

ENG-ASYNC - Asynchronous execution

In our customer's words:
”A slow partner must not hold the whole product hostage or lose work.”

What your boss defined as done:

- ✓ Workflow execution continues outside the request that triggered it, which answers before the work is finished.
- ✓ An existing broker or job queue carries pending work, and the choice is justified.
- ✓ A worker separate from the application server can be stopped, work queued, then restarted without losing that work.
- ✓ Third party waits are bounded by documented timeouts.
- ✓ Worker capacity can be changed through configuration rather than source modification.

ENG-EXT - Extensible connectors

In our customer's words:
”A new integration should not require rewriting the engine.”

What your boss defined as done:

- ✓ Node implementations respect a common, documented interface covering validation and execution.
- ✓ Node types register at start up, isolated from workflow execution logic, with no dynamic loading of third party code.
- ✓ Adding the last node type did not change the engine: its commit touches only the new node and its registration.
- ✓ **HOWTOCONTRIBUTE.md** contains a reproducible procedure, and a trivial node type is added from it during the defense.
- ✓ A new **Action** or **REAction** appears in **about.json** without a second hard coded catalogue nor a change to the payload shape.



ENG-RETRY - Retries and idempotency

In our customer's words:
”Transient errors should recover without performing the same side effect four times.”

What your boss defined as done:

- ✓ Failed nodes are retried automatically a bounded number of times with growing delays, stated before the evaluator provokes a failure.
- ✓ Retries do not require a broker, and this requirement never requires `ENG-ASYNC`.
- ✓ A side effecting **REAction** is protected against duplicate effects during retry, and the mechanism is explained.
- ✓ Exhausted work is retained in a dead letter store from which it can be inspected and replayed.
- ✓ The evaluator removes the cause of failure and replays a dead lettered run successfully.
- ✓ The retry policy, attempts and delays are configuration.
- ✓ The group explains how retry, persistence and idempotence depend on each other in its architecture.

ENG-TEMPLATES - Shareable templates

In our customer's words:
”I want to share an automation without sharing my credentials.”

What your boss defined as done:

- ✓ A user exports an **AREA** from the web client as a single portable file containing no credentials or tokens.
- ✓ Another user imports it and binds it to their own **service** subscriptions.
- ✓ A missing **service** subscription is reported by name, and nothing is created.
- ✓ A tampered or structurally invalid definition is rejected at import, and nothing is created.
- ✓ The format is documented and versioned, including the policy for older versions.

SEC-AUTH - Authentication hardening

The common base asks authentication and delegated authorization to work. This requirement asks the group to demonstrate abuse resistance.

In our customer's words:
”Nobody must abuse our login callback, and a bot must not try passwords all night without resistance or evidence.”

What your boss defined as done:

- ✓ Accepted redirect targets follow an exact allow list, documented with who controls it, and a close but invalid target is rejected.

- ✓ An unsolicited or replayed callback is rejected, and the group explains what distinguishes the second attempt.
- ✓ Authorization code interception is mitigated, and the group explains the mechanism and its limits rather than naming it.
- ✓ The same protections are demonstrated on the mobile login, not only in the browser.
- ✓ Registration and authentication are rate limited with documented limits.
- ✓ Exceeding a limit returns the dedicated HTTP status and a retry signal, observed by replaying the attack loop used before the limit existed.
- ✓ A legitimate user is never locked out permanently, and the recovery behavior is stated.
- ✓ Limited attempts are recorded with address and time but never with passwords, and at least two indicators of such an attack are documented.

SEC-CRYPTO - Encryption at rest

In our customer's words:
”Assume an attacker obtains the database. Limit what the dump reveals.”

What your boss defined as done:

- ✓ The group shows what a database dump exposed before encryption and identifies the sensitive data.
- ✓ Third party tokens are encrypted at rest with an authenticated cipher, and the evaluator reads ciphertext in the database.
- ✓ The key lives outside the source, the repository and the database, and its provisioning is documented.
- ✓ No token, in clear or encrypted, appears in the logs of a full **AREA** run.
- ✓ Key rotation is documented, including what happens to data encrypted with the previous key.
- ✓ A tampered token is detected and refused when the evaluator alters one byte.

SEC-DEPS - Dependency scanning

In our customer's words:
”Show that vulnerable dependencies become visible and actionable.”

What your boss defined as done:

- ✓ Dependency vulnerability scanning runs automatically on every contribution across the server and both clients, through a pipeline, a commit hook or a scheduled job.
- ✓ A stated severity threshold blocks contributions, shown in the automation history or provoked live with a known vulnerable dependency.
- ✓ The current report is reachable, and the group explains its most severe open finding.
- ✓ Accepted findings record rationale and compensating measures.
- ✓ The group explains how a newly disclosed vulnerability reaches its attention.

SEC-GDPR - Personal data compliance

In our customer's words:
”A user must be able to know what personal data we hold, export it and have it deleted without a developer.”

What your boss defined as done:

- ✓ A committed data inventory maps personal data to purpose, storage location and retention.
- ✓ An authenticated user exports their data from the web client in a machine readable file.
- ✓ Account deletion leaves no personal data in the database beyond what is documented as deliberately kept, with its justification.
- ✓ Account deletion revokes or destroys delegated third party tokens.
- ✓ Retention of execution records is stated and technically enforced.
- ✓ What deletion means for backups is documented: rewritten, expired within a stated delay, or replayed on restore.

SEC-SSRF - SSRF protection

In our customer's words:
”User controlled outbound requests must not reach internal targets.”

What your boss defined as done:

- ✓ Before the protection exists, the group demonstrates the attack against a target it controls and states the potential impact.
- ✓ User supplied addresses in private, loopback and reserved ranges are rejected, and the evaluator tries at least three.
- ✓ A target resolving differently at check time and at connection time is blocked, and the mechanism and its limits are documented.
- ✓ Redirects are not followed, or are revalidated with the same rules, and the choice is documented.
- ✓ Accepted schemes follow a documented allow list.
- ✓ A blocked call leaves a record an operator can inspect.
- ✓ Outbound calls are bounded by a stated timeout and maximum response size, and the evaluator sees a slow or oversized target stopped.

SEC-THREAT - Threat modeling

In our customer's words:
”Explain a credible attack chain, what we would observe, and what stops it.”

What your boss defined as done:

- ✓ A threat model covers workflow execution and outbound calls, not only login, with a stated method

naming assets, entry points and trust boundaries.
- ✓ One plausible attack chain is documented step by step, from initial access to impact, in the vocabulary of a recognized attack framework.
- ✓ Each step states its mitigation and whether it is implemented today.
- ✓ Each step identifies at least one detection opportunity: what an operator sees, and where.
- ✓ The group plays the whole chain against its controlled environment and keeps evidence of each step.
- ✓ At least two detection indicators are implemented, and replaying the chain raises them where documented.

SYS-BACKUP - Backup and restore

In our customer's words:
”Do not tell me you have backups; restore one.”

What your boss defined as done:

- ✓ Timestamped database backups run on a schedule and are stored outside the database container.
- ✓ A single documented command or script restores a backup.
- ✓ The evaluator destroys data and sees a successful restore, with the product working on the previous data.
- ✓ The worst case data loss is stated and follows from the schedule.
- ✓ Third party tokens in backups are encrypted, or the documentation states what protects them.

SYS-HEALTH - Health checks

In our customer's words:
”Tell me which dependency is failing.”

What your boss defined as done:

- ✓ A machine readable HTTP health endpoint reports each dependency separately: at least the database, the third party authentication providers and whatever runs the **hooks**.
- ✓ A required dependency failure turns the answer into a failing status.
- ✓ The evaluator stops the database container and sees it identified.
- ✓ Checks are bounded, so the endpoint answers within one second even when a dependency is unreachable.
- ✓ No secret, connection string or stack trace is exposed.

SYS-K8S - Container orchestration

In our customer's words:
”Run the product on an orchestrated cluster and survive the loss of one application instance.”

What your boss defined as done:

- ✓ The *Kubernetes* manifests for the whole product are versioned in a dedicated directory, covering deployments, services, entry point, configuration and secrets.
- ✓ The product runs on a local or managed cluster during the defense, and the evaluator reaches the web client through its entry point.
- ✓ The server runs with multiple replicas and keeps answering when the evaluator deletes one.
- ✓ Configuration and secrets come from cluster objects, not from images or container files. This requirement carries its own minimal probes and secret wiring, and never requires `SYS-HEALTH` or `SYS-SECRETS`.
- ✓ The group diagnoses a deliberately broken component with the cluster command line.

SYS-OBS - Observability

In our customer's words:
”I need to understand a workflow execution after it happened and see the product's behavior over time.”

What your boss defined as done:

- ✓ Every workflow run writes a persisted record: **AREA**, user, timing, nodes run with their inputs and outputs, and outcome.
- ✓ Finished runs are never rewritten, and the enforcement is explained.
- ✓ Run count, failure rate and duration are exposed over HTTP in a documented format that a monitoring tool of the market reads without adaptation.
- ✓ A provisioned dashboard comes up with the stack, populated after a few runs, with no manual setup.
- ✓ From the web client alone, the evaluator learns how much the product ran over a period and which **AREA** fails most.
- ✓ The history, the metrics and the web views read one single store of execution records.

SYS-RELEASE - Release automation

The common base requires automated checks and deployment. This requirement extends it to versioned, traceable and reversible releases.

In our customer's words:
”I want to know exactly which version is running, and get yesterday's back in minutes when today's breaks.”

What your boss defined as done:

- ✓ Every integration into the main branch builds the server and web client images once, tags them with



the commit they come from, and publishes them to a registry, and the evaluator pulls one by its tag.
- ✓ Pushing a version tag produces a release by automation, referencing its published images, carrying the installable Android application, and listing a changelog generated from the history since the previous release.
- ✓ The deployment runs the published images and never builds on its target, and the evaluator matches the version reported by the running product to a release and its commit.
- ✓ A deployment whose post-deployment check fails is stopped and the previous release is put back without human action, shown in the automation history or provoked live with a deliberately broken release. This check never requires `SYS-HEALTH`.
- ✓ A single documented command or action redeploys any earlier release, and the evaluator does it.
- ✓ A live status badge in the **README.md** shows the state of the main branch and leads to its latest run.

SYS-SECRETS - Secrets management

In our customer's words:
”Prove secrets do not leak through source, images or logs.”

What your boss defined as done:

- ✓ Secret scanning over current content and history is clean, or every historical secret is documented as revoked, with the proof.
- ✓ A complete `.env.example` documents every required variable with a description and a harmless placeholder.
- ✓ A fresh clone configured from the example file starts a running product, and the evaluator does it.
- ✓ Secrets reach containers through the environment or an orchestrator secret mechanism, never through committed files or baked images.
- ✓ No secret appears in any service log, and the evaluator greps a full run.



v 6.4.0

